1. Inheritance in Java

➤ Concept

- **Inheritance** allows one class to **reuse** the code of another.
- The class being inherited from is the **superclass**.
- The class that inherits is the **subclass**.

Syntax:

```
class Parent {
    // Parent field and method
}
class Child extends Parent {
     // Child field and method
}
```

📊 IS-A Relationship:

```
Child IS-A Parent
```

➤ Inherited Members

When a subclass is created using `extends`, it automatically gets most of the members (fields and methods) of its superclass, except some special cases.

Member Type	Inherited?	Notes
Fields (instance variables)	✅ Yes	But access depends on the **access modifier** (e.g., `private` not directly accessible).
Instance methods	✅ Yes	Can be used or **overridden** in the subclass.
Private members	✅ Yes, but ⚠️ Not directly accessible	They exist in the object but are **hidden**; accessible only through **public methods** of the superclass.
Constructors	❌ No	Constructors are **never inherited**. A subclass must call one using `super()`.

```
class Parent {
    public String a = "a";
    private String b = "b";
    public String getB(){
        return this.b;
    }
    private void doubleB(){
        this.b = this.b + this.b;
    }
}

class Child extends Parent {
    private String c = "c";
    public void print(){
        this.a; // visible
        this.b; // not visible
        this.getB(); // visible
        this.doubleB(); // not visible
    }
}
```

➤ Constructor Calls and Inheritance

When you create a subclass object, the constructor of the parent class is always called first.

If you do not explicitly call a parent constructor using `super()`,
the compiler automatically inserts a call to the no-argument parent constructor (super();) as the first line of the subclass constructor.

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    // No constructor explicitly written
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}
```

even though Child has no constructor, Java automatically add

```
Child() {
    super(); // automatically inserted
}
```

If the parent class does NOT have a no-argument constructor, and the subclass does not explicitly call super(args), you will get a compile-time error.

```
class Parent {
    Parent(int x) { }
}

class Child extends Parent {
    // ❌ Compiler error: Parent() does not exist
}
```

---

2. Multiple Inheritance

- Java **does not support** multiple inheritance using classes to avoid ambiguity.

Example problem:

```
class A { void show() {} }
class B { void show() {} }
class C extends A, B { }  // ❌ Error
```

Use **interfaces** for multiple inheritance (will be covered later).

🤝 3. Aggregation (HAS-A Relationship)

- When one class has a reference to another class.

Example:

```
class Address {
    String city;
}

class Student {
    Address addr; // HAS-A relationship
}
```

Use **inheritance** for **IS-A**, and **aggregation** for **HAS-A**.

🔄 4. Method Overriding

- Same method name and parameters, but **different implementation** in subclass.

Example:

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
    void sound() { System.out.println("Bark"); }
}
```

➤ Overloading VS. Overriding

Overloading → "Same name, different forms" → happens in **same class** → compile-time resolution

Overriding → "**Subclass** `replaces` parent behavior" → happens across classes → runtime resolution

🧩 **Static methods cannot be overridden.**

🧬 5. Covariant Return Type

- Since Java 5, subclass methods can override a parent method and return a **subtype**.

Example:

```
class Animal {}
class Dog extends Animal {}

class AnimalShelter {
    Animal getAnimal() { return new Animal(); }
}

class DogShelter extends AnimalShelter {
    Dog getAnimal() { return new Dog(); }  // ✅ Covariant return type
}
```

🧰 6. `super` Keyword

Used to refer to the **parent class**.

Examples:

```
super.varName;          // Access parent variable
super.methodName();     // Call parent method
super();                // Call parent constructor
```

The parent constructor runs **before** the child's constructor.

⚙️ 7. Instance Initializer Block

- Runs each time an object is created, **after the parent constructor**.

Example:

```
class Parent {

    // Instance initializer block
    {
        System.out.println("Parent Instance Block");
    }

    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {

    // Instance initializer block
    {
        System.out.println("Child Instance Block");
    }

    Child() {
        System.out.println("Child Constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}
```

output:

```
Parent Instance Block
Parent Constructor
Child Instance Block
Child Constructor
```

🔒 10. The `final` Keyword

Used to **restrict modification**:

- `final class` → cannot be extended.
- `final method` → cannot be overridden.
- `final variable` → value cannot change.

Example:

```
final class A {}
class B extends A {}   // ❌ Error
```

- A final variable means:

You can `assign it only once`, and then it cannot change.

Type	When initialized
final instance/non-static variable	during object creation (declaration or constructor or init block)
final static variable	during class loading
final local variable	before first use in method